
Algoritmos de Busca

Dos fundamentos à Engenharia de Software



Prof. Dr. João Choma

UEM

github.com/JoaoChoma



Ao final desta aula, espera-se que você seja capaz de:

- compreender o problema fundamental de **busca**;
- diferenciar estratégias de busca;
- analisar custo computacional;
- compreender buscas em estruturas lineares e grafos;
- compreender buscas informadas por heurísticas;
- reconhecer problemas de busca na Engenharia de Software;
- compreender recuperação de informação aplicada a artefatos de software;
- compreender o conceito de **Search-Based Software Engineering (SBSE)**;
- relacionar algoritmos clássicos com problemas reais de desenvolvimento de software.



O que significa buscar?

Um problema aparentemente simples



Imagine um vetor:

```
[12, 7, 31, 4, 18, 25, 9, 42, 3]
```

Queremos encontrar:

```
25
```

Uma solução simples:

```
12 → não  
7  → não  
31 → não  
4  → não  
18 → não  
25 → ENCONTREI !
```

Acabamos de executar um algoritmo de busca.



Problemas de busca aparecem quando queremos:

- encontrar um elemento;
- encontrar um caminho;
- encontrar uma solução;
- encontrar uma configuração;
- encontrar o melhor resultado;
- encontrar algo suficientemente bom;
- recuperar informação relevante.



De forma geral:

ENTRADA



Espaço de possibilidades



Estratégia de busca



Critério de avaliação



SOLUÇÃO

Pergunta fundamental:

Em qual ordem devemos explorar as possibilidades?



Chamamos de **espaço de busca** o conjunto de todas as possíveis soluções que podem ser consideradas.

Exemplo:

Senha de 4 dígitos

0000

0001

0002

...

9999

Espaço de busca:

4

$10^4 = 10.000$ possibilidades



Imagine uma senha com 20 caracteres.

Ou:

- todas as rotas possíveis entre cidades;
- todas as configurações possíveis de módulos;
- todas as sequências possíveis de testes;
- todas as combinações possíveis de requisitos para uma release.

O problema deixa de ser apenas:

"Existe uma solução?"

e passa a ser:

"Como encontrar uma solução sem explorar tudo?"



Nesta aula veremos:

ALGORITMOS DE BUSCA | —

Busca em estruturas |

├ Busca linear |

└ Busca binária | —

Busca em grafos |

├ BFS |

└ DFS | —

Busca informada |

├ Greedy Best-First |

└ A* | —

Busca por otimização

├ Hill Climbing

├ Simulated Annealing

└ Algoritmos Genéticos



Busca Linear



A estratégia mais simples possível:

Verifique cada elemento até encontrar o procurado.

```
[15, 22, 8, 19, 31, 4]
```

```
buscar 19
```

Execução:

```
15 x  
22 x  
8 x  
19 ✓
```



```
buscaLinear(lista, alvo):  
    para i de 0 até tamanho(lista)-1:  
        se lista[i] == alvo:  
            retorne i  
    retorne -1
```



Melhor caso:

$O(1)$

Elemento está na primeira posição.

Pior caso:

$O(n)$

Precisamos percorrer toda a estrutura.



Busca linear é interessante quando:

- os dados não estão ordenados;
- o conjunto é pequeno;
- fazemos poucas buscas;
- ordenar os dados teria custo desnecessário.



Busca Binária



Considere:

```
[3, 7, 12, 18, 25, 31, 42, 55, 61]
```

Queremos encontrar:

```
42
```

Como os dados estão ordenados, podemos eliminar grandes partes do espaço de busca.



Dividir o espaço pela metade.

```
3  7  12  18  [25] 31  42  55  61
```

$42 > 25$.

Logo:

```
eliminamos toda a metade esquerda.
```



Restou:

31 42 55 61

Dividimos novamente.

Cada comparação reduz aproximadamente:

$n \downarrow$

$n/2 \downarrow$

$n/4 \downarrow$

$n/8 \downarrow$

...



Busca linear:

$O(n)$

Busca binária:

$O(\log n)$



Busca linear:

até 1.000.000.000 comparações

Busca binária:

aproximadamente 30 comparações

Porque:

$$2^{\log(1.000.000.000)} \approx 30$$



A busca binária não é rápida apenas porque seu código é "melhor".
Ela é rápida porque utiliza uma propriedade do problema:

ORDENAÇÃO

Essa ideia aparecerá novamente:

Conhecimento sobre o problema pode reduzir drasticamente o espaço de busca.



Busca em Grafos



Imagine:



Queremos encontrar:

H

Qual caminho devemos seguir?



Um grafo pode representar:

- cidades e estradas;
- páginas e hyperlinks;
- pessoas e relacionamentos;
- classes e dependências;
- módulos de software;
- chamadas entre métodos;
- commits e branches;
- requisitos e artefatos;
- estados possíveis de um sistema.



Breadth-First Search
BFS
Busca em Largura

e

Depth-First Search
DFS
Busca em Profundidade



A BFS explora primeiro os elementos mais próximos.



Ordem possível:

A → B → C → D → E → F



BFS utiliza uma:

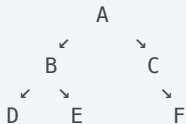
FILA

FIFO:

First In
First Out



Imagine ondas se propagando:



Primeiro distância 0.

Depois distância 1.

Depois distância 2.

E assim por diante.



Em grafos não ponderados, BFS pode encontrar:

o caminho com menor número de arestas.

Por isso aparece em problemas de:

- caminhos mínimos;
- redes;
- dependências;
- navegação;
- análise de relacionamentos.



A DFS segue um caminho o máximo possível antes de retornar.



Uma execução possível:

A↓
B↓
D↑
B↓
E↑
A↓
C



DFS utiliza:

PILHA

ou implicitamente:

RECURSÃO



Característica	BFS	DFS
Estrutura	Fila	Pilha
Estratégia	largura	profundidade
Caminho mínimo não ponderado	Sim	Não necessariamente
Memória	pode ser alta	geralmente menor
Backtracking	não é característica central	natural
Exploração profunda	lenta	eficiente



Se queremos descobrir:

"Existe alguma dependência entre o módulo A e o módulo X?"

DFS pode ser suficiente.

Mas se queremos:

"Qual é o menor caminho de dependências entre A e X?"

BFS passa a ser particularmente interessante.



Busca Informada



BFS e DFS sabem:

onde estou

e:

onde posso ir

Mas não necessariamente sabem:

qual direção parece melhor.

E se tivermos alguma informação adicional?



Uma **heurística** estima quão promissor é determinado estado.
Representamos frequentemente como:

$h(n)$

onde:

$h(n)$ = estimativa do custo de n até o objetivo



Queremos chegar a Curitiba.
Temos duas cidades possíveis:

Cidade A → 80 km em linha reta de Curitiba
Cidade B → 300 km em linha reta de Curitiba

Uma heurística poderia priorizar:

Cidade A



Estratégia:

Explore primeiro o estado que parece estar mais próximo do objetivo.

Avaliação:

$$f(n) = h(n)$$

Problema:

A opção que **parece** melhor agora pode levar a um caminho ruim depois.



O algoritmo A* combina:

custo já percorrido
+
estimativa do custo restante

Formalmente:

$$f(n) = g(n) + h(n)$$

onde:

$g(n)$ = custo do início até n
 $h(n)$ = estimativa de n até o objetivo



O A* pergunta:

Quanto já gastei?

+

Quanto provavelmente ainda vou gastar?



Compare:

BFS
"Vou explorar sistematicamente."

Greedy
"Vou para onde parece melhor."

A*
"Vou considerar o que já custou
e o que ainda parece custar."



Busca como Otimização



Até agora:

Existe uma solução?

Agora:

Qual é a MELHOR solução?

Temos então um problema de:

OTIMIZAÇÃO



Precisamos escolher um conjunto de casos de teste.
Existem:

10.000 testes

Mas só temos tempo para executar:

500

Queremos:

maximizar cobertura
minimizar tempo
maximizar detecção de defeitos

Qual conjunto escolher?



Podemos transformar a qualidade de uma solução em uma função:

```
fitness(solução)
```

Exemplo simplificado:

```
fitness =  
    cobertura * 0.6  
    - tempo * 0.2  
    + defeitosDetectados * 0.2
```

Agora podemos procurar soluções que maximizem:

```
fitness
```



Ideia:

```
solução atual
    ↓
gera vizinhos
    ↓
escolhe um melhor
    ↓
repete
```

É como subir uma montanha procurando sempre um ponto mais alto.



Podemos chegar a um:

ótimo local

e acreditar que encontramos o melhor resultado.



Permite, ocasionalmente:

aceitar uma solução pior

Por quê?

Para escapar de ótimos locais.

No início:

maior exploração

Depois:

maior estabilização



Inspirados em evolução.
Mantemos uma:

POPULAÇÃO DE SOLUÇÕES

e executamos:

```
avaliação  
  ↓  
seleção  
  ↓  
crossover  
  ↓  
mutação  
  ↓  
nova geração
```



```
Geração 1
  ↓
seleção dos melhores
  ↓
recombinação
  ↓
mutação
  ↓
Geração 2
  ↓
...
  ↓
soluções melhores
```



Estudamos:

- Busca Linear
- Busca Binária
- BFS
- DFS
- Greedy
- A*
- Hill Climbing
- Simulated Annealing
- Algoritmos Genéticos

Mas...

O que isso tem a ver com Engenharia de Software?



Busca na Engenharia de Software



Uma mudança de perspectiva

Durante o desenvolvimento de software buscamos constantemente.

Buscamos:

- código;
- requisitos;
- dependências;
- bugs;
- testes;
- commits;
- desenvolvedores;
- componentes;
- soluções arquiteturais;
- configurações;
- refatorações.



Considere:

"Quais classes são afetadas se eu modificar esta classe?"

Isso pode ser tratado como:

BUSCA EM GRAFO



Pergunta:

O que pode ser impactado por uma alteração em GatewayPagamento?

Estamos diante de um problema de exploração de grafo.



Podemos representar chamadas entre métodos:

```
main()  
|  
+--> autenticar()  
|  
+--> processarPedido()  
    |  
    +--> validar()  
    |  
    +--> calcularTotal()  
    |  
    +--> salvar()
```

DFS/BFS podem apoiar diferentes análises dessa estrutura.



Imagine:

Requisito



Classe



Método



Outro método



Banco

Ao modificar um requisito queremos descobrir:

quais artefatos podem ser afetados?

Isso é uma forma de:

CHANGE IMPACT ANALYSIS



Code Search



Quantas vezes um desenvolvedor procura:

```
"onde fazemos autenticação?"
```

ou:

```
"onde calculamos desconto?"
```

ou simplesmente:

```
calculateDiscount
```

Buscar código é uma atividade central do desenvolvimento.



Uma abordagem simples procura correspondências textuais.

Consulta:

```
authenticate
```

Código:

```
public User authenticate(...)
```

Excelente.

Mas e se o código possuir:

```
public User login(...)
```

?



Usuário procura:

```
authenticate user
```

Código possui:

```
login()
```

Semanticamente:

muito próximos

Lexicalmente:

diferentes

Esse problema motiva técnicas mais sofisticadas.



A busca em software pode utilizar técnicas de:

Information Retrieval — IR

Em vez de simplesmente perguntar:

"é igual?"

podemos perguntar:

"quão relevante é?"



Documentos e consultas podem ser transformados em vetores.

Exemplo conceitual:

consulta:

"validar pagamento cartão"

↓

[0.7, 0.1, 0.8, 0.2, ...]

↑

código/documento:

[0.6, 0.2, 0.9, 0.1, ...]

Calculamos uma medida de similaridade.



Uma técnica clássica é:

TF-IDF

Term Frequency:

quanto o termo aparece neste documento?

Inverse Document Frequency:

quanto esse termo é discriminativo na coleção?



Podemos comparar dois vetores:



Quanto menor o ângulo:

maior a similaridade



Agora a busca não retorna simplesmente:

```
encontrado  
não encontrado
```

Ela pode retornar:

1. PaymentValidator.java	0.94
2. CreditCardService.java	0.87
3. CheckoutController.java	0.72
4. CustomerService.java	0.21

Temos um:

RANKING DE RELEVÂNCIA



Modelos modernos podem representar:

linguagem natural

e:

código

em espaços vetoriais capazes de capturar relações semânticas.

Consulta:

"função que verifica se o usuário pode acessar o sistema"

pode recuperar:

```
boolean hasPermission(User user)
```

mesmo sem palavras idênticas.



Uma visão simplificada:

```
grep / string matching
  ↓
indexação
  ↓
Information Retrieval
  ↓
modelos estruturais
  ↓
Machine Learning
  ↓
Deep Learning
  ↓
Transformers / embeddings
  ↓
LLMs
```



Uma revisão ampla de Code Search descreve décadas de técnicas envolvendo:

- diferentes tipos de consultas;
- pré-processamento;
- expansão de consultas;
- indexação;
- recuperação;
- ranking;
- poda de resultados;
- técnicas semânticas.

Referência: Di Grazia & Pradel.



Rastreabilidade como Busca



Considere:

REQ-17
"Permitir recuperação de senha"

Precisamos descobrir relações com:

REQ-17
|
+--> PasswordController
|
+--> PasswordService
|
+--> EmailService
|
+--> ResetPasswordTest



Problema:

Dado um requisito, quais artefatos estão relacionados a ele?

Podemos interpretar isso como um problema de:

RECUPERAÇÃO DE INFORMAÇÃO



Podemos comparar:

REQUISITO



CASO DE USO



CÓDIGO



TESTES

e produzir candidatos:

Requisito → Classe A 0.91

Requisito → Classe B 0.83

Requisito → Classe C 0.17

O engenheiro pode então revisar os candidatos.



Rastreabilidade apoia:

- manutenção;
- evolução;
- análise de impacto;
- conformidade;
- compreensão do sistema;
- gerenciamento de mudanças.



Busca em Repositórios de Software



Um projeto contém:

- Código
- +
- Commits
- +
- Issues
- +
- Pull Requests
- +
- Testes
- +
- Autores
- +
- Datas
- +
- Branches

Esse conjunto registra parte da história do desenvolvimento.



MSR utiliza dados provenientes de repositórios para investigar fenômenos relacionados ao desenvolvimento e evolução de software.

Perguntas possíveis:

Quais arquivos mudam juntos?

Quais módulos concentram defeitos?

Quem possui maior experiência neste módulo?

Quais commits estão relacionados a determinado bug?



Imagine o histórico:

```
Commit 1 → A.java + B.java  
Commit 2 → A.java + B.java  
Commit 3 → A.java + C.java  
Commit 4 → A.java + B.java
```

Podemos descobrir:

```
A.java ↔ B.java
```

possuem forte relação histórica.

Mesmo que uma dependência explícita não seja evidente no código.



Search-Based Software Engineering



Em 2001, Harman e Jones propuseram uma perspectiva importante:

Muitos problemas de Engenharia de Software podem ser reformulados como problemas de busca.

Essa área ficou conhecida como:

Search-Based Software Engineering

SBSE



Pegamos um problema de Engenharia de Software:

```
selecionar testes
```

e o transformamos em:

```
Espaço de busca  
+  
Representação  
+  
Função de fitness  
+  
Algoritmo de busca
```



PROBLEMA DE ES



Representação das soluções



Espaço de busca



Função de fitness



Metaheurística



Solução candidata



Problema:

Temos 5.000 testes.

Só podemos executar 500.

Objetivos:

↑
cobertura↑
probabilidade de detectar falhas↓
tempo



Uma solução poderia ser:

```
[1,0,0,1,1,0,0,1,...]
```

onde:

```
1 = executar teste  
0 = não executar
```

Temos então uma solução candidata.



Exemplo:

```
fitness(T) =  
    0.5 × cobertura(T)  
+ 0.3 × históricoFalhas(T)  
- 0.2 × tempo(T)
```

Agora temos algo que um algoritmo pode:

OTIMIZAR



Onde SBSE pode ser aplicada?

Exemplos incluem:

- geração de casos de teste;
- seleção de casos de teste;
- priorização de testes;
- localização de defeitos;
- refatoração;
- arquitetura;
- modularização;
- planejamento de releases;
- estimativa;
- gerenciamento de requisitos;
- manutenção;
- configuração de sistemas.



Queremos gerar automaticamente entradas capazes de executar:

```
if (idade >= 18 && saldo > 1000) {  
    aprovar();  
}
```

Uma função de fitness pode avaliar:

Quão próxima determinada entrada está de executar o ramo desejado?

O algoritmo começa então a procurar entradas melhores.



Imagine uma arquitetura com:

alto acoplamento
baixa coesão

Queremos encontrar uma organização melhor.
Uma solução representa:

distribuição das classes entre módulos

Fitness poderia considerar:

↑
coesão↓
acoplamento



Temos requisitos:

R1 R2 R3 R4 R5 ... R100

Cada requisito possui:

valor
custo
risco
dependências
prioridade

Temos recursos limitados.

Pergunta:

Qual conjunto de requisitos deve entrar na próxima release?

Isso é um problema de otimização.



Engenharia de Software raramente possui apenas um objetivo.
Queremos simultaneamente:

↓
custo↓
tempo↓
risco↑

qualidade↑
cobertura↑
valor

Esses objetivos podem entrar em conflito.



Considere:

Solução A
Custo: 10
Qualidade: 80

Solução B
Custo: 15
Qualidade: 95

Qual é melhor?

Depende dos objetivos e dos trade-offs envolvidos.



Em otimização multiobjetivo podemos procurar soluções:

Qualidade $\uparrow$



----- Custo

Nenhuma dessas soluções é necessariamente melhor em todos os objetivos.
O engenheiro escolhe considerando os trade-offs.



Um mapa geral



Técnica	Exemplo em Engenharia de Software
Busca Linear	localizar artefato em coleção pequena
Busca Binária	consultas em estruturas ordenadas
DFS	explorar dependências
BFS	menor cadeia de dependências
IR	localizar código/requisitos
Busca semântica	code search por significado
Hill Climbing	otimização de configuração
Simulated Annealing	problemas combinatórios
Algoritmos Genéticos	testes, arquitetura, planejamento
Multiobjetivo	releases, testes, arquitetura



Existem pelo menos três ideias diferentes de "busca" em ES:

1. EXPLORAÇÃO
percorrer estruturas
2. RECUPERAÇÃO
encontrar informação relevante
3. OTIMIZAÇÃO
encontrar boas soluções



Exemplo:

Quais métodos podem ser alcançados a partir deste método?

Ferramentas:

BFS
DFS
grafos



Exemplo:

Qual trecho de código implementa "recuperação de senha"?

Ferramentas:

Information Retrieval
TF-IDF
similaridade
embeddings
modelos semânticos



Exemplo:

Qual subconjunto de testes maximiza cobertura dentro de 30 minutos?

Ferramentas:

- Hill Climbing
- Simulated Annealing
- Genetic Algorithms
- Multi-objective Search



Desafio para a turma



Uma empresa possui:

```
2 milhões de linhas de código  
18.000 commits  
4.500 issues  
12.000 testes  
800 requisitos
```

Um desenvolvedor recebe:

"O cálculo de desconto para clientes premium está errado."



Como encontrar:

onde o desconto é implementado?

Possíveis abordagens:

busca textual
code search
Information Retrieval
busca semântica



Como descobrir:

quais componentes dependem desse código?

Possíveis abordagens:

```
call graph  
dependency graph  
BFS  
DFS
```



Como descobrir:

quais testes devem ser executados?

Podemos utilizar:

dependências
histórico
rastreabilidade
impact analysis



Existem 12.000 testes.

Temos apenas:

20 minutos

Quais executar primeiro?

Agora temos um problema de:

OTIMIZAÇÃO



BUG REPORT



Information Retrieval



LOCALIZAÇÃO DO CÓDIGO



Busca em grafo



ANÁLISE DE IMPACTO



Rastreabilidade



TESTES RELACIONADOS



Busca/otimização



PRIORIZAÇÃO DOS TESTES



Algoritmos de busca não são apenas:

"achar um número dentro de um vetor"

Eles são mecanismos para navegar:

DADOS
ESTADOS
GRAFOS
DOCUMENTOS
CÓDIGO
ARTEFATOS
SOLUÇÕES



Muitos problemas podem ser reinterpretados como:

O QUE ESTOU PROCURANDO?↓

QUAL É O ESPAÇO DE BUSCA?↓

COMO REPRESENTO UMA SOLUÇÃO?↓

COMO AVALIO UMA SOLUÇÃO?↓

QUAL ESTRATÉGIA DE BUSCA É ADEQUADA?



Classifique cada problema como predominantemente:

EXPLORAÇÃO

RECUPERAÇÃO

OTIMIZAÇÃO

- 1 Encontrar classes que dependem de `PagamentoService`.
- 2 Encontrar código relacionado a "cancelamento de pedido".
- 3 Escolher 200 testes entre 10.000.
- 4 Encontrar requisitos relacionados a determinada classe.
- 5 Descobrir o menor caminho entre dois módulos.
- 6 Selecionar requisitos para a próxima release.
- 7 Encontrar commits relacionados a um bug.
- 8 Reorganizar classes para reduzir acoplamento.



Problema	Categoria
Dependências de PagamentoService	Exploração
Código de cancelamento	Recuperação
Seleção de testes	Otimização
Requisito → classe	Recuperação
Caminho entre módulos	Exploração
Release planning	Otimização
Bug → commits	Recuperação
Redução de acoplamento	Otimização



Busca não é um algoritmo.

>

Busca é uma classe de problemas.

A estratégia adequada depende:

- do espaço de busca;
- do conhecimento disponível;
- do tamanho do problema;
- da qualidade exigida;
- do custo computacional;
- do objetivo da busca.



CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C.
Introduction to Algorithms.
MIT Press.



RUSSELL, S.; NORVIG, P.

Artificial Intelligence: A Modern Approach.

Pearson.

Referência importante para:

- busca em espaço de estados;
- busca não informada;
- busca informada;
- heurísticas;
- A*;
- otimização.



HARMAN, M.; JONES, B. F.

Search-Based Software Engineering.

Information and Software Technology, v. 43, n. 14, p. 833–839, 2001.

DOI: 10.1016/S0950-5849(01)00189-6

Trabalho seminal da área de SBSE.



- HARMAN, M.; JONES, B. F. *Search-Based Software Engineering*. Information and Software Technology, 2001. DOI: 10.1016/S0950-5849(01)00189-6.
- DI GRAZIA, L.; PRADEL, M. *Code Search: A Survey of Techniques for Finding Code*.
- XIE, Y. et al. *Survey of Code Search Based on Deep Learning*. ACM Transactions on Software Engineering and Methodology, 2024.
- CORMEN, T. H. et al. *Introduction to Algorithms*. MIT Press.
- RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. Pearson.



Quando encontrar um problema de Engenharia de Software, pergunte:

"Será que eu consigo transformar isso em um problema de busca?"